

Summary: Diffusion Models for MNIST Generation

GitHub repository: <https://github.com/jlvi179/Advanced-Deep-Learning>

Key Concept

This notebook implements a Denoising Diffusion Probabilistic Model (DDPM) to generate handwritten digits (MNIST). Rather than focusing on a basic implementation from scratch, the notebook utilizes the highly optimized `denoising_diffusion_pytorch` library, demonstrating practical tweaks required to stabilize and accelerate diffusion model training.

Crucial Tweaks and Optimizations

Training generative diffusion models is notoriously resource-intensive. The following architectural and procedural tweaks were necessary to make training feasible:

- **U-Net Dimensionality Scaling:** To prevent the model from becoming too large to fit in memory or train in a reasonable time, the base feature dimension was restricted to `dim = 32`, with a specific multiplier scaling of `dim_mults = (1, 2, 5)`. This creates a heavily compressed bottleneck suitable for 28×28 grayscale images (`channels = 1`).
- **Disabling Flash Attention:** By default, modern transformer/U-Net architectures attempt to use Flash Attention for speed. However, this often leads to compatibility issues on systems without specific CUDA setups or dedicated GPUs. Explicitly setting `flash_attn = False` guarantees the code can run across varied hardware environments (like standard Colab or local CPUs).
- **Accelerated DDIM Sampling:** The forward diffusion process uses $T = 1000$ steps to smoothly add noise. During inference, stepping back 1000 times is exceedingly slow. By setting `sampling_timesteps = 250`, the model implicitly utilizes DDIM (Denoising Diffusion Implicit Models) sampling, heavily accelerating generation by skipping intermediate timesteps without significantly degrading image quality.
- **Weight Decay via AdamW:** Diffusion models are prone to overfitting to the specific noise patterns in smaller datasets. Using AdamW instead of a standard Adam optimizer introduces decoupled weight decay, which helps regularize the U-Net parameters smoothly.
- **Progressive Evaluation Visualization:** Because full training requires hours of compute (often exceeding free Colab tiers), a critical workflow tweak is the inline sampling at regular intervals. Using `make_grid`, visual outputs are checked dynamically, providing qualitative feedback on convergence long before the final epoch is reached.